

2.05-hpi-lasso-dev

September 2, 2024

```
[114]: %load_ext autoreload
      %autoreload 2

import pandas as pd
import numpy as np
import torch
# import seaborn as sns
# from sklearn.linear_model import LogisticRegression

from prism.config import PROCESSED_DATA_DIR, MODELS_DIR
from prism.legacy import normalise, modelStats
from prism.maskedmlp import train_mlp_batched, mlpmask_pytorch
from prism.save_models import save_mlp
from prism.lasso import lasso
from prism.partial_responses import partial_responses
from prism.nomogram import nomogram
```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

```
[115]: %reload_ext autoreload
```

```
[30]: # Parameters
device = 'cpu'
method = 'dirac'
SAVE_MODELS = False
seed = 257

np.random.seed(seed)
torch.manual_seed(seed)
```

```
[30]: <torch._C.Generator at 0x19fae6a6fd0>
```

1 Import data

```
[31]: data_train = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_train.
↳csv'))
data_test = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_test.
↳csv'))
data_val = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_val.csv'))

data_train_test = pd.concat([data_train, data_test])

# Drop id column
data_train.drop('trr_id_code', axis=1, inplace=True)
data_test.drop('trr_id_code', axis=1, inplace=True)
data_val.drop('trr_id_code', axis=1, inplace=True)

target_col = 'oneyearmort'

x_train0 = data_train.drop(target_col, axis=1)
y_train = data_train[target_col]

x_test0 = data_test.drop(target_col, axis=1)
y_test = data_test[target_col]

x_val0 = data_val.drop(target_col, axis=1)
y_val = data_val[target_col]

[x_train, x_test] = normalise(x_train0, x_test0)
x_val = normalise(x_val0)

x_train_tensor = torch.tensor(x_train.values, dtype=torch.float32,
↳device=device)
y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32)
x_test_tensor = torch.tensor(x_test.values, dtype=torch.float32, device=device)
y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32)
x_val_tensor = torch.tensor(x_val.values, dtype=torch.float32, device=device)
y_val_tensor = torch.tensor(y_val.values, dtype=torch.float32)
```

```
[32]: feature_names = [
    'don age',
    'don isch time min',
    'rec age yr',
    'rec creat',
    'rec infect 2wk',
    'rec vent',
    'rec sex',
    'tx year',
    'ICM',
```

```
'NICM',  
'prior tx'  
]
```

2 Train MLP

```
[33]: mlp_params = {  
    'n_hidden': 10,  
    'weight_decay': 1e-5,  
    'lr': 0.001,  
    'patience': 50,  
    'tolerance': 0.0001,  
    'batch_size': 1024,  
    'device': device,  
    'seed': seed  
}  
  
mlp_batched = train_mlp_batched(x_train, y_train, x_test, y_test, **mlp_params)
```

```
Epoch 0: Train loss 0.6690, Val loss 0.6166  
Epoch 1: Train loss 0.5749, Val loss 0.5303  
Epoch 2: Train loss 0.5062, Val loss 0.4680  
Epoch 3: Train loss 0.4573, Val loss 0.4224  
Epoch 4: Train loss 0.4240, Val loss 0.3906  
Epoch 5: Train loss 0.4014, Val loss 0.3693  
Epoch 6: Train loss 0.3869, Val loss 0.3563  
Epoch 7: Train loss 0.3782, Val loss 0.3485  
Epoch 8: Train loss 0.3726, Val loss 0.3440  
Epoch 9: Train loss 0.3694, Val loss 0.3416  
Epoch 10: Train loss 0.3667, Val loss 0.3402  
Epoch 11: Train loss 0.3653, Val loss 0.3395  
Epoch 12: Train loss 0.3644, Val loss 0.3393  
Epoch 13: Train loss 0.3640, Val loss 0.3392  
Epoch 14: Train loss 0.3631, Val loss 0.3391  
Epoch 15: Train loss 0.3621, Val loss 0.3391  
Epoch 16: Train loss 0.3624, Val loss 0.3392  
Epoch 17: Train loss 0.3617, Val loss 0.3392  
Epoch 18: Train loss 0.3613, Val loss 0.3393  
Epoch 19: Train loss 0.3611, Val loss 0.3392  
Epoch 20: Train loss 0.3611, Val loss 0.3393  
Epoch 21: Train loss 0.3608, Val loss 0.3394  
Epoch 22: Train loss 0.3601, Val loss 0.3392  
Epoch 23: Train loss 0.3604, Val loss 0.3392  
Epoch 24: Train loss 0.3604, Val loss 0.3392  
Epoch 25: Train loss 0.3601, Val loss 0.3390  
Epoch 26: Train loss 0.3598, Val loss 0.3390  
Epoch 27: Train loss 0.3593, Val loss 0.3389
```

Epoch 28: Train loss 0.3598, Val loss 0.3391
Epoch 29: Train loss 0.3586, Val loss 0.3388
Epoch 30: Train loss 0.3588, Val loss 0.3389
Epoch 31: Train loss 0.3587, Val loss 0.3388
Epoch 32: Train loss 0.3584, Val loss 0.3389
Epoch 33: Train loss 0.3585, Val loss 0.3388
Epoch 34: Train loss 0.3584, Val loss 0.3388
Epoch 35: Train loss 0.3588, Val loss 0.3387
Epoch 36: Train loss 0.3583, Val loss 0.3386
Epoch 37: Train loss 0.3581, Val loss 0.3386
Epoch 38: Train loss 0.3583, Val loss 0.3385
Epoch 39: Train loss 0.3576, Val loss 0.3384
Epoch 40: Train loss 0.3577, Val loss 0.3384
Epoch 41: Train loss 0.3574, Val loss 0.3385
Epoch 42: Train loss 0.3572, Val loss 0.3384
Epoch 43: Train loss 0.3567, Val loss 0.3383
Epoch 44: Train loss 0.3573, Val loss 0.3385
Epoch 45: Train loss 0.3568, Val loss 0.3383
Epoch 46: Train loss 0.3569, Val loss 0.3384
Epoch 47: Train loss 0.3573, Val loss 0.3383
Epoch 48: Train loss 0.3570, Val loss 0.3380
Epoch 49: Train loss 0.3573, Val loss 0.3382
Epoch 50: Train loss 0.3567, Val loss 0.3381
Epoch 51: Train loss 0.3571, Val loss 0.3381
Epoch 52: Train loss 0.3568, Val loss 0.3379
Epoch 53: Train loss 0.3562, Val loss 0.3378
Epoch 54: Train loss 0.3567, Val loss 0.3377
Epoch 55: Train loss 0.3564, Val loss 0.3379
Epoch 56: Train loss 0.3566, Val loss 0.3380
Epoch 57: Train loss 0.3558, Val loss 0.3378
Epoch 58: Train loss 0.3564, Val loss 0.3379
Epoch 59: Train loss 0.3559, Val loss 0.3376
Epoch 60: Train loss 0.3550, Val loss 0.3377
Epoch 61: Train loss 0.3555, Val loss 0.3378
Epoch 62: Train loss 0.3561, Val loss 0.3378
Epoch 63: Train loss 0.3547, Val loss 0.3377
Epoch 64: Train loss 0.3551, Val loss 0.3377
Epoch 65: Train loss 0.3556, Val loss 0.3378
Epoch 66: Train loss 0.3553, Val loss 0.3375
Epoch 67: Train loss 0.3555, Val loss 0.3377
Epoch 68: Train loss 0.3552, Val loss 0.3376
Epoch 69: Train loss 0.3547, Val loss 0.3377
Epoch 70: Train loss 0.3555, Val loss 0.3376
Epoch 71: Train loss 0.3549, Val loss 0.3378
Epoch 72: Train loss 0.3546, Val loss 0.3376
Epoch 73: Train loss 0.3546, Val loss 0.3376
Epoch 74: Train loss 0.3550, Val loss 0.3375
Epoch 75: Train loss 0.3541, Val loss 0.3375

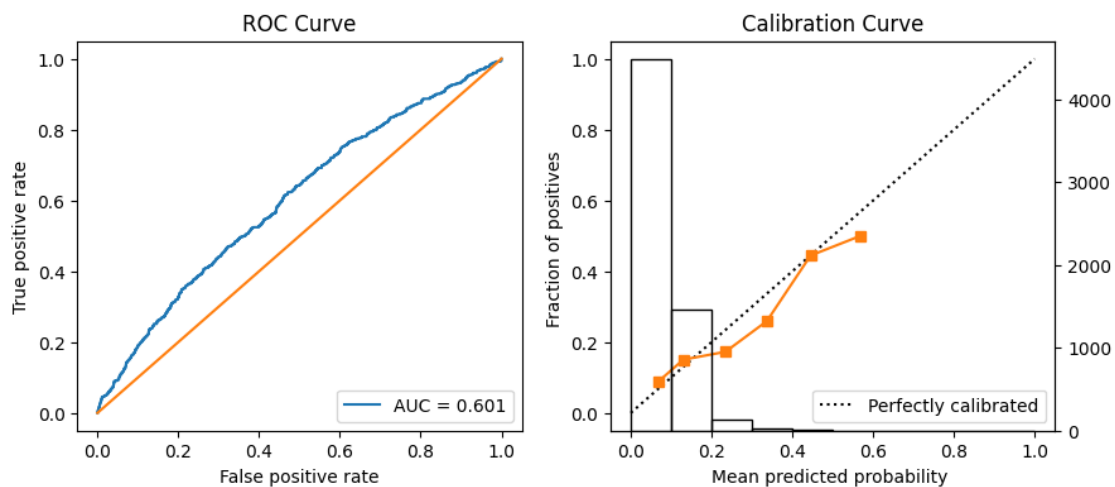
Epoch 76: Train loss 0.3547, Val loss 0.3378
Epoch 77: Train loss 0.3543, Val loss 0.3376
Epoch 78: Train loss 0.3545, Val loss 0.3375
Epoch 79: Train loss 0.3542, Val loss 0.3374
Epoch 80: Train loss 0.3552, Val loss 0.3375
Epoch 81: Train loss 0.3544, Val loss 0.3374
Epoch 82: Train loss 0.3538, Val loss 0.3376
Epoch 83: Train loss 0.3546, Val loss 0.3375
Epoch 84: Train loss 0.3540, Val loss 0.3374
Epoch 85: Train loss 0.3552, Val loss 0.3374
Epoch 86: Train loss 0.3548, Val loss 0.3376
Epoch 87: Train loss 0.3542, Val loss 0.3373
Epoch 88: Train loss 0.3540, Val loss 0.3376
Epoch 89: Train loss 0.3542, Val loss 0.3375
Epoch 90: Train loss 0.3542, Val loss 0.3376
Epoch 91: Train loss 0.3540, Val loss 0.3375
Epoch 92: Train loss 0.3543, Val loss 0.3375
Epoch 93: Train loss 0.3537, Val loss 0.3374
Epoch 94: Train loss 0.3539, Val loss 0.3377
Epoch 95: Train loss 0.3543, Val loss 0.3375
Epoch 96: Train loss 0.3540, Val loss 0.3374
Epoch 97: Train loss 0.3537, Val loss 0.3377
Epoch 98: Train loss 0.3535, Val loss 0.3374
Epoch 99: Train loss 0.3537, Val loss 0.3374
Epoch 100: Train loss 0.3534, Val loss 0.3375
Epoch 101: Train loss 0.3542, Val loss 0.3376
Epoch 102: Train loss 0.3539, Val loss 0.3376
Epoch 103: Train loss 0.3540, Val loss 0.3377
Epoch 104: Train loss 0.3532, Val loss 0.3373
Epoch 105: Train loss 0.3537, Val loss 0.3378
Epoch 106: Train loss 0.3537, Val loss 0.3375
Epoch 107: Train loss 0.3535, Val loss 0.3374
Epoch 108: Train loss 0.3535, Val loss 0.3377
Epoch 109: Train loss 0.3533, Val loss 0.3377
Epoch 110: Train loss 0.3535, Val loss 0.3377
Epoch 111: Train loss 0.3536, Val loss 0.3376
Epoch 112: Train loss 0.3539, Val loss 0.3379
Epoch 113: Train loss 0.3533, Val loss 0.3376
Epoch 114: Train loss 0.3539, Val loss 0.3377
Epoch 115: Train loss 0.3536, Val loss 0.3379
Epoch 116: Train loss 0.3543, Val loss 0.3379
Epoch 117: Train loss 0.3534, Val loss 0.3376
Epoch 118: Train loss 0.3534, Val loss 0.3376
Epoch 119: Train loss 0.3545, Val loss 0.3378
Epoch 120: Train loss 0.3538, Val loss 0.3379
Epoch 121: Train loss 0.3538, Val loss 0.3374
Epoch 122: Train loss 0.3535, Val loss 0.3374
Epoch 123: Train loss 0.3536, Val loss 0.3377

```
Epoch 124: Train loss 0.3527, Val loss 0.3378
Epoch 125: Train loss 0.3535, Val loss 0.3378
Epoch 126: Train loss 0.3532, Val loss 0.3377
Epoch 127: Train loss 0.3538, Val loss 0.3380
Epoch 128: Train loss 0.3536, Val loss 0.3378
Epoch 129: Train loss 0.3537, Val loss 0.3378
Epoch 130: Train loss 0.3527, Val loss 0.3378
Epoch 131: Train loss 0.3532, Val loss 0.3376
Epoch 132: Train loss 0.3534, Val loss 0.3377
Epoch 133: Train loss 0.3533, Val loss 0.3378
Epoch 134: Train loss 0.3529, Val loss 0.3377
Epoch 135: Train loss 0.3537, Val loss 0.3379
Epoch 136: Train loss 0.3535, Val loss 0.3380
Stopping early at epoch 87
```

3 Evaluate MLP

```
[34]: y_test_blackbox = mlp_batched.predict(x_test_tensor).cpu().numpy()
      y_val_blackbox = mlp_batched.predict(x_val_tensor).cpu().numpy()

      mlp_metrics_test = modelStats(y_test_blackbox, y_test, y_train, ROC=True,
      ↪mdlCalibration=True, metricNames=True, auc_ci=True)
      modelStats(y_val_blackbox, y_val, y_train, ROC=True, mdlCalibration=True,
      ↪metricNames=True, auc_ci=True);
```

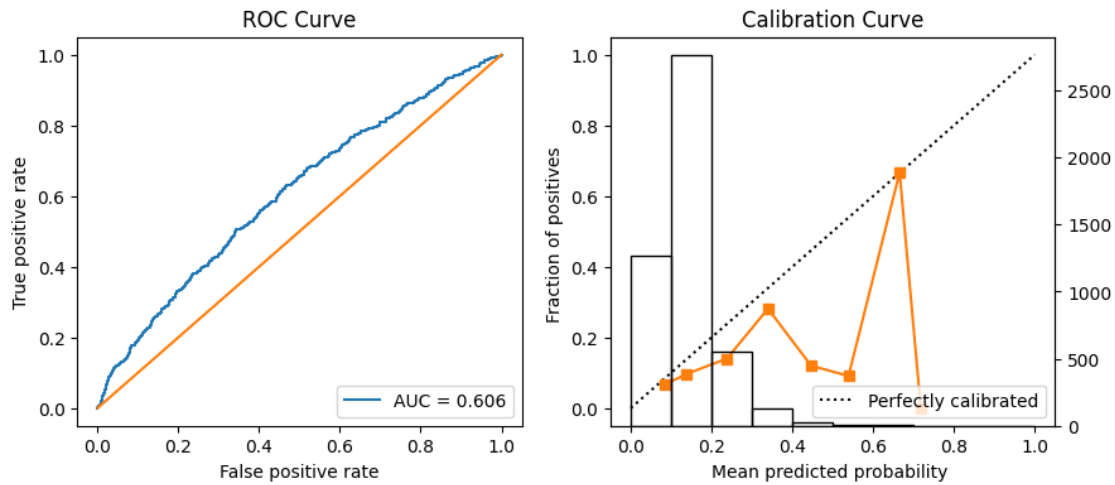


```
----- Metrics -----
prevalence: 0.123
sensitivity: 0.241
specificity: 0.864
```

```

accuracy: 0.797
ppv: 0.175
auc score: 0.601
auc lower ci: 0.577
auc upper ci: 0.623
-----

```



```

----- Metrics -----
prevalence: 0.123
sensitivity: 0.667
specificity: 0.489
accuracy: 0.507
ppv: 0.126
auc score: 0.606
auc lower ci: 0.582
auc upper ci: 0.631
-----

```

```

[34]: {'prevalence': 0.123,
      'sensitivity': 0.667,
      'specificity': 0.489,
      'accuracy': 0.507,
      'ppv': 0.126,
      'auc score': 0.606,
      'auc lower ci': '0.582',
      'auc upper ci': '0.631'}

```

```

[35]: # Save MLP
      if SAVE_MODELS:
          save_mlp(mlp_batched, mlp_params, mlp_metrics_test, MODELS_DIR)

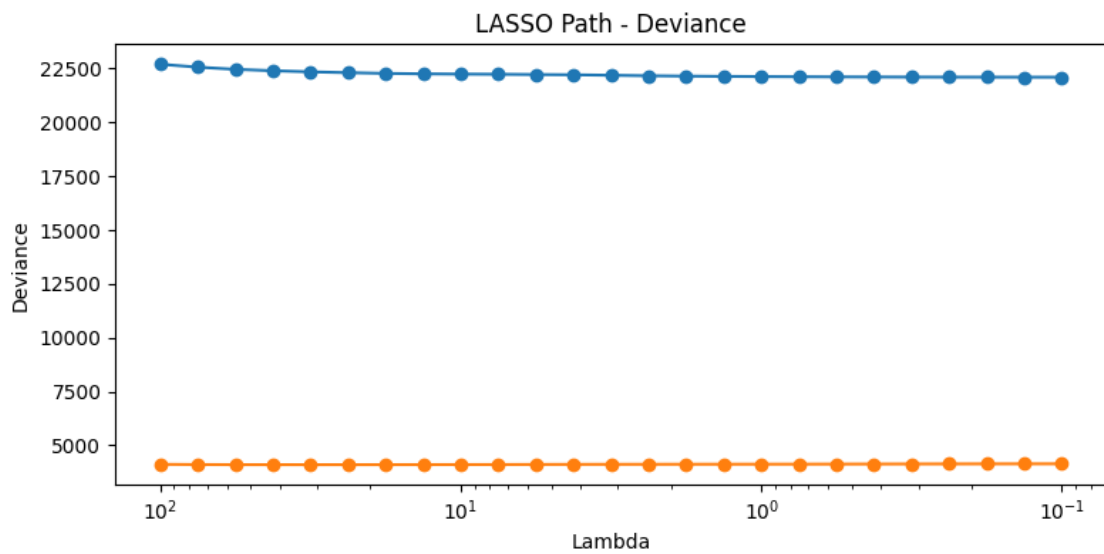
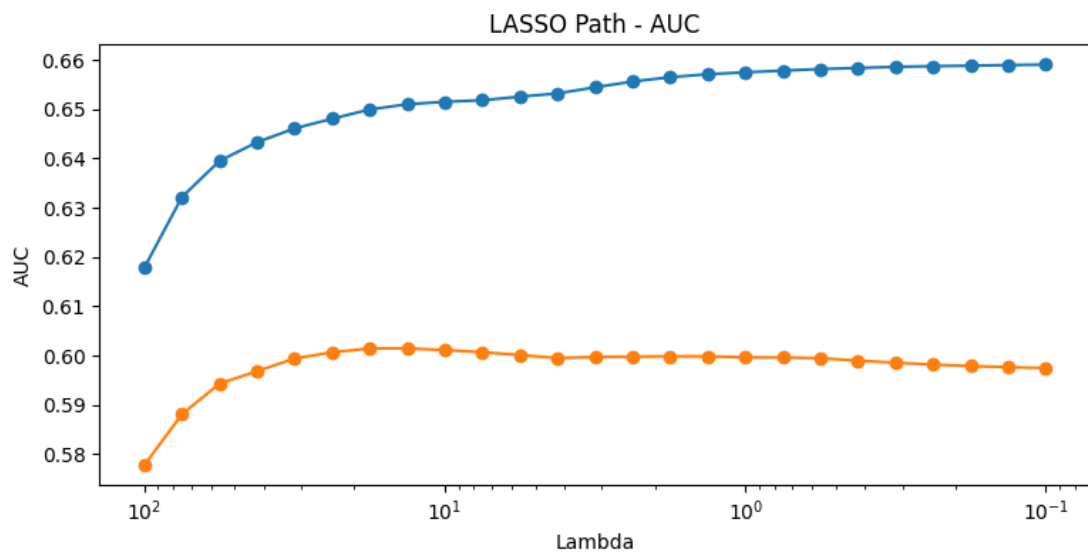
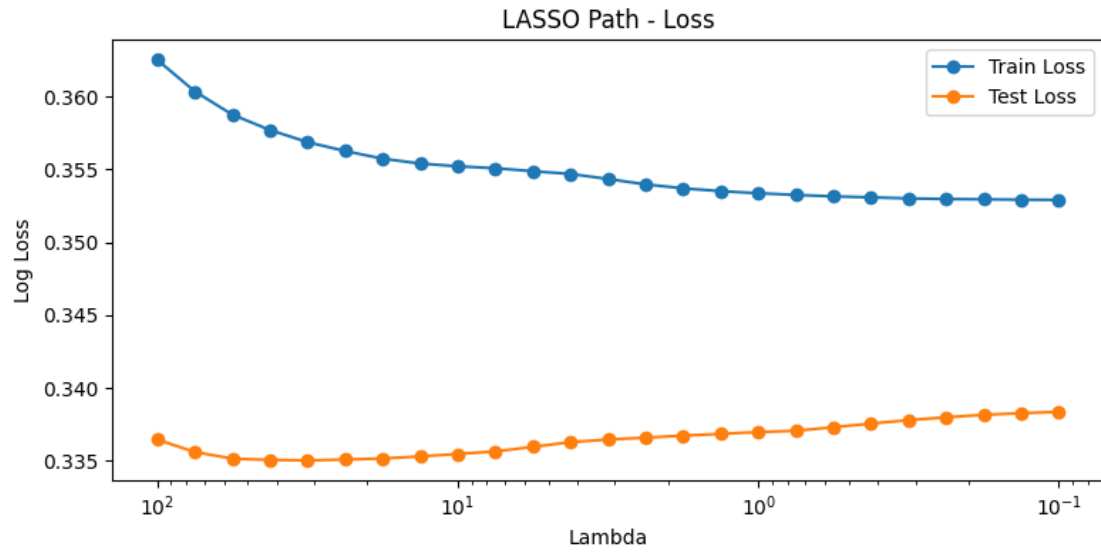
```

4 MLP LASSO

```
[36]: partial_responses_train, partial_responses_test, bivariate_inputs = ↳
↳partial_responses(
    x_train_tensor,
    x_test_tensor,
    mlp_batched,
    method=method,
    device=device
)
```

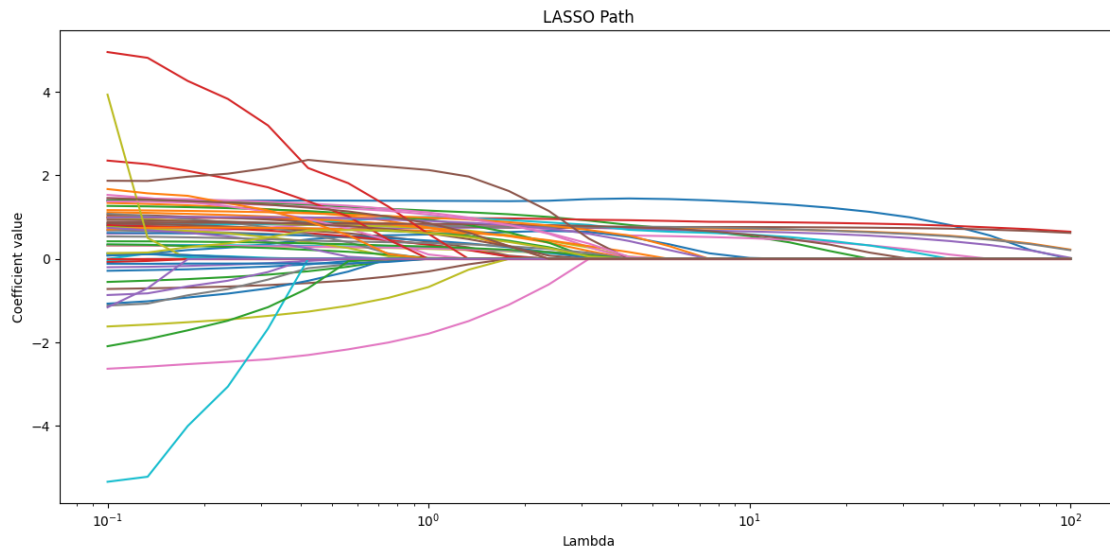
5 Run LASSO on MLP Partial Responses

```
[93]: %reload_ext autoreload
lasso_results = lasso(
    partial_responses_train,
    partial_responses_test,
    y_train,
    y_test,
    bivariate_inputs,
    feature_names=feature_names,
    nlambda=25,
    min_lambda=0.1,
    max_lambda=100,
    tol=1e-4,
    batch_size=2
)
```

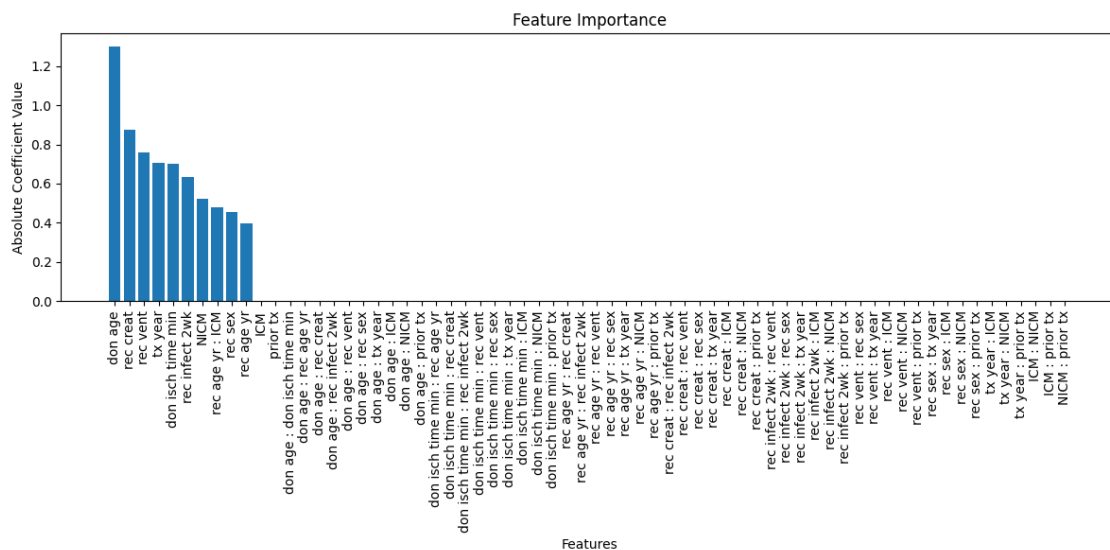



```
<IPython.core.display.HTML object>
```

```
[94]: lasso_results.plot_lambda_path()
```

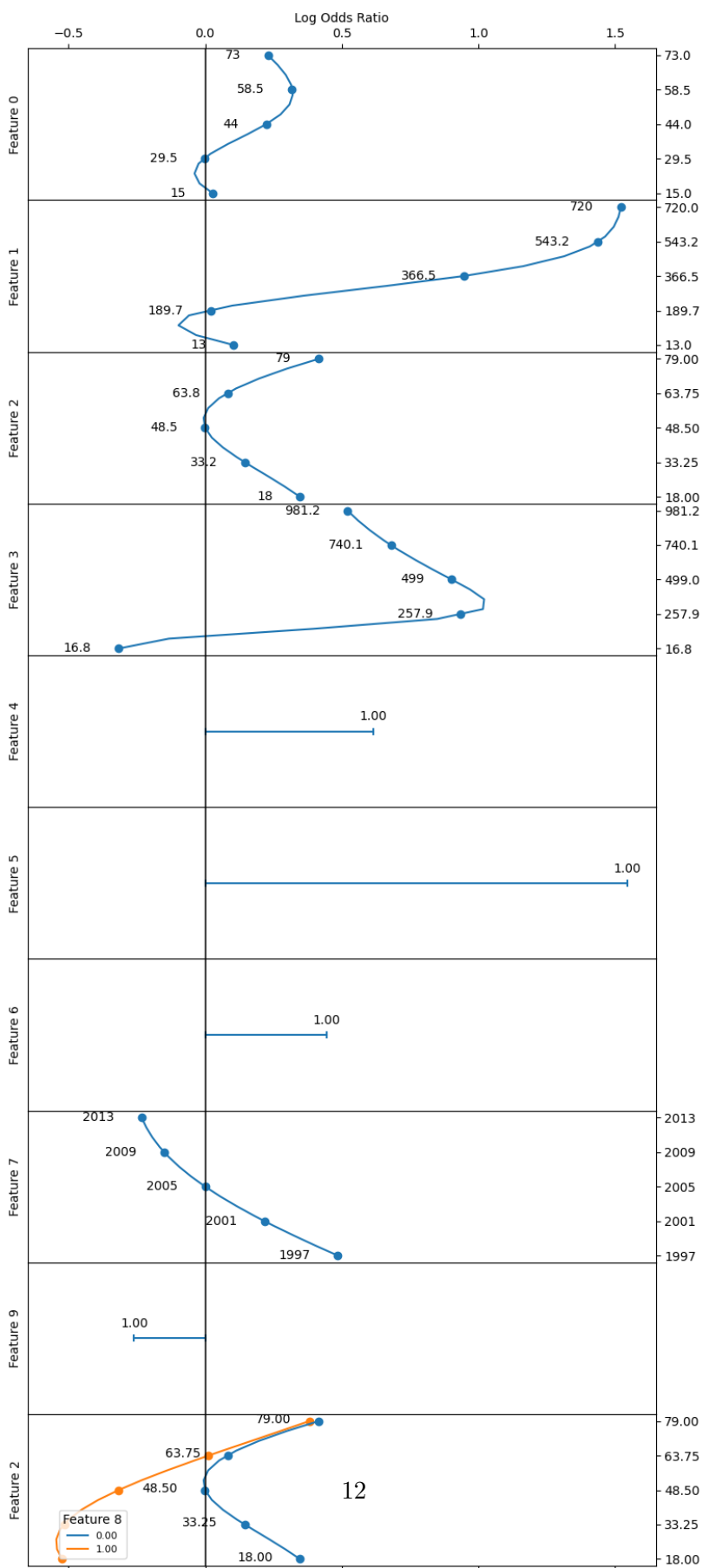


```
[71]: lasso_results.select_lambda(7)
lasso_results.plot_feature_importance()
```



```
[40]: %reload_ext autoreload
# Note: The prPlots function is not yet available in the new implementation.
# We'll use the new nomogram function instead.
nomogram_results = nomogram(
    lasso_results,
    x_train_tensor,
    x_train0.median().values,
    x_train0.std().values,
    mlp_batched,
    n_steps=15,
    sd_scale=2,
    method=method,
    device=device,
    categorical_threshold=15,
    subtract_univariate=False
);
```

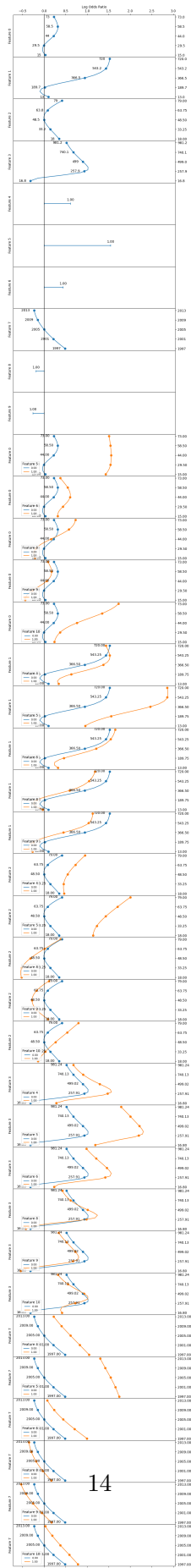
Nomogram of univariate and mixed bivariate partial responses

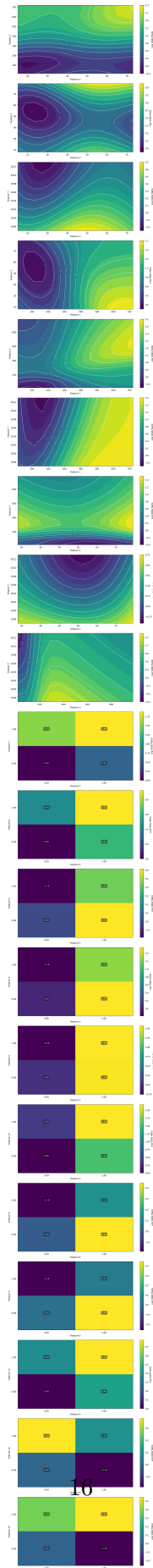


```
[41]: # test with more features
lasso_results.select_lambda(24)
```

```
[42]: %reload_ext autoreload
nomogram_results = nomogram(
    lasso_results,
    x_train_tensor,
    x_train0.median().values,
    x_train0.std().values,
    mlp_batched,
    n_steps=15,
    sd_scale=2,
    method=method,
    device=device,
    categorical_threshold=15,
    subtract_univariate=False
)
```

Nonogram of univariate and mixed bivariate partial responses



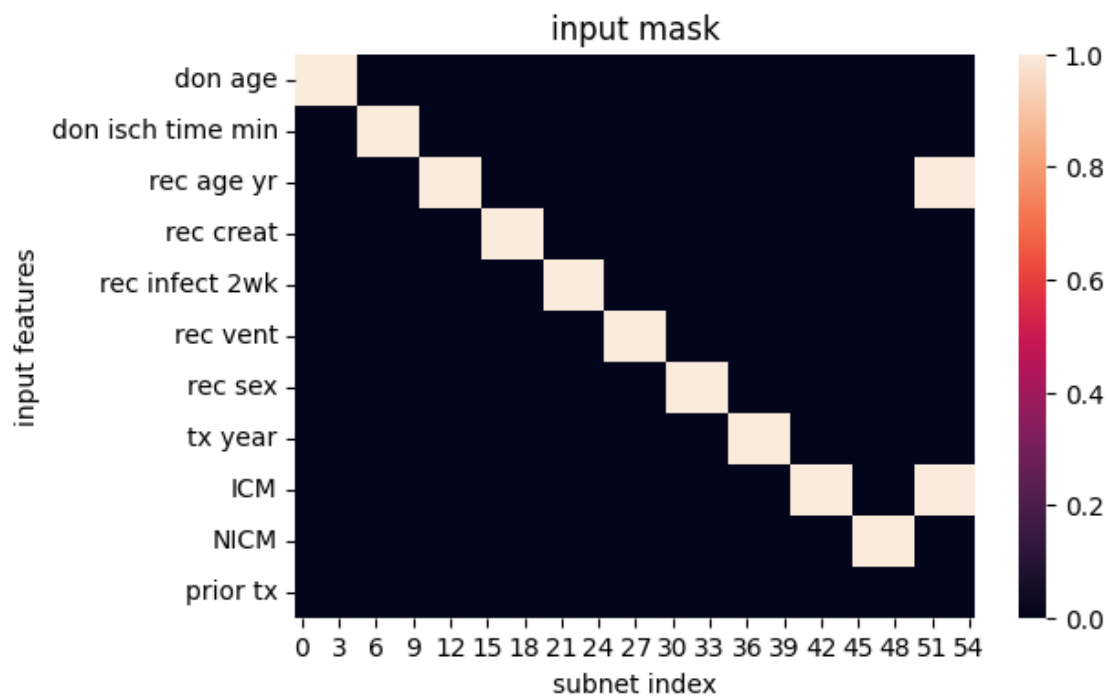



```
[97]: # return to previously selected lambda
lasso_results.select_lambda(7)
```

6 Train the Partial Response Network

```
[102]: mask, n_features = lasso_results.get_mask()
```

Active features: ['don age', 'don isch time min', 'rec age yr', 'rec creat', 'rec infect 2wk', 'rec vent', 'rec sex', 'tx year', 'NICM', 'rec age yr : ICM']



```
[103]: prn_params = {
    'n_hidden': n_features,
    'mask': mask,
    'subnet_nodes': 5,
    'iter': 10000,
    'lr': 0.05,
    'weight_decay': 0.00001,
    'tolerance': 0.0001,
    'patience': 100,
    'device': device,
    'seed': seed
}
```

```
}
```

```
prn = mlpmask_pytorch(x_train, y_train, x_test, y_test, **prn_params)
```

```
Epoch 0, Training loss 0.7491949796676636, Validation loss 0.5058018565177917
Epoch 1, Training loss 0.5208706855773926, Validation loss 0.3845042586326599
Epoch 2, Training loss 0.4087001085281372, Validation loss 0.34338873624801636
Epoch 3, Training loss 0.3777748942375183, Validation loss 0.3566133677959442
Epoch 4, Training loss 0.3964877128601074, Validation loss 0.38072437047958374
Epoch 5, Training loss 0.4208233952522278, Validation loss 0.39131689071655273
Epoch 6, Training loss 0.427925169467926, Validation loss 0.38494935631752014
Epoch 7, Training loss 0.41638851165771484, Validation loss 0.36732038855552673
Epoch 8, Training loss 0.3941255211830139, Validation loss 0.3479166626930237
Epoch 9, Training loss 0.3729674220085144, Validation loss 0.33749744296073914
Epoch 10, Training loss 0.3656228482723236, Validation loss 0.34155094623565674
Epoch 11, Training loss 0.37698012590408325, Validation loss 0.3493284583091736
Epoch 12, Training loss 0.390849232673645, Validation loss 0.34701046347618103
Epoch 13, Training loss 0.3884521424770355, Validation loss 0.3386439383029938
Epoch 14, Training loss 0.37404561042785645, Validation loss 0.33528995513916016
Epoch 15, Training loss 0.3619023859500885, Validation loss 0.34104466438293457
Epoch 16, Training loss 0.35944950580596924, Validation loss 0.35184338688850403
Epoch 17, Training loss 0.3643118143081665, Validation loss 0.36123892664909363
Epoch 18, Training loss 0.37016406655311584, Validation loss 0.36496368050575256
Epoch 19, Training loss 0.3723222315311432, Validation loss 0.36210665106773376
Epoch 20, Training loss 0.3696381747722626, Validation loss 0.3544863164424896
Epoch 21, Training loss 0.3639782965183258, Validation loss 0.3455081284046173
Epoch 22, Training loss 0.3587774634361267, Validation loss 0.3386857509613037
Epoch 23, Training loss 0.35707536339759827, Validation loss 0.33580368757247925
Epoch 24, Training loss 0.35936906933784485, Validation loss 0.3357851803302765
Epoch 25, Training loss 0.3628600239753723, Validation loss 0.33608561754226685
Epoch 26, Training loss 0.36388930678367615, Validation loss 0.33564338088035583
Epoch 27, Training loss 0.3615776300430298, Validation loss 0.335593044757843
Epoch 28, Training loss 0.3582203686237335, Validation loss 0.3372500240802765
Epoch 29, Training loss 0.3564668297767639, Validation loss 0.3404138386249542
Epoch 30, Training loss 0.3570193946361542, Validation loss 0.3435826897621155
Epoch 31, Training loss 0.35869941115379333, Validation loss 0.3451804220676422
Epoch 32, Training loss 0.35986778140068054, Validation loss 0.3445168435573578
Epoch 33, Training loss 0.3596443831920624, Validation loss 0.3420269787311554
Epoch 34, Training loss 0.3582829236984253, Validation loss 0.3389121890068054
Epoch 35, Training loss 0.35680246353149414, Validation loss 0.33643555641174316
Epoch 36, Training loss 0.35620635747909546, Validation loss 0.33520403504371643
Epoch 37, Training loss 0.3567262887954712, Validation loss 0.33492323756217957
Epoch 38, Training loss 0.3576492965221405, Validation loss 0.33491653203964233
Epoch 39, Training loss 0.35799235105514526, Validation loss 0.3349255919456482
Epoch 40, Training loss 0.3574346601963043, Validation loss 0.3352886140346527
Epoch 41, Training loss 0.3565181791782379, Validation loss 0.33637866377830505
Epoch 42, Training loss 0.3559926152229309, Validation loss 0.3380572199821472
Epoch 43, Training loss 0.3561197817325592, Validation loss 0.3396962583065033
```

Epoch 44, Training loss 0.356579065322876, Validation loss 0.3406146168708801
Epoch 45, Training loss 0.35686594247817993, Validation loss 0.3404918313026428
Epoch 46, Training loss 0.3567216396331787, Validation loss 0.33949875831604004
Epoch 47, Training loss 0.35627955198287964, Validation loss 0.3381355106830597
Epoch 48, Training loss 0.35589656233787537, Validation loss 0.3369191288948059
Epoch 49, Training loss 0.3558361530303955, Validation loss 0.3361293077468872
Epoch 50, Training loss 0.3560543954372406, Validation loss 0.33577653765678406
Epoch 51, Training loss 0.356268048286438, Validation loss 0.3357725143432617
Epoch 52, Training loss 0.356241375207901, Validation loss 0.33608710765838623
Epoch 53, Training loss 0.35600021481513977, Validation loss 0.33671995997428894
Epoch 54, Training loss 0.35576489567756653, Validation loss 0.3375648558139801
Epoch 55, Training loss 0.35571014881134033, Validation loss 0.3383626341819763
Epoch 56, Training loss 0.3558168113231659, Validation loss 0.338813841342926
Epoch 57, Training loss 0.3559291958808899, Validation loss 0.33874815702438354
Epoch 58, Training loss 0.3559182286262512, Validation loss 0.33821582794189453
Epoch 59, Training loss 0.35578784346580505, Validation loss 0.33744338154792786
Epoch 60, Training loss 0.35564911365509033, Validation loss 0.3366982340812683
Epoch 61, Training loss 0.35560452938079834, Validation loss 0.3361610174179077
Epoch 62, Training loss 0.35565441846847534, Validation loss 0.3358895778656006
Epoch 63, Training loss 0.35570958256721497, Validation loss 0.33587130904197693
Epoch 64, Training loss 0.35568973422050476, Validation loss 0.336080938577652
Epoch 65, Training loss 0.35560184717178345, Validation loss 0.3364782929420471
Epoch 66, Training loss 0.35551899671554565, Validation loss 0.3369692265987396
Epoch 67, Training loss 0.35549667477607727, Validation loss 0.3374041020870209
Epoch 68, Training loss 0.3555222451686859, Validation loss 0.3376360833644867
Epoch 69, Training loss 0.35554012656211853, Validation loss 0.3375965356826782
Epoch 70, Training loss 0.35551342368125916, Validation loss 0.33732742071151733
Epoch 71, Training loss 0.35545578598976135, Validation loss 0.3369502127170563
Epoch 72, Training loss 0.3554099202156067, Validation loss 0.3365987241268158
Epoch 73, Training loss 0.3554005026817322, Validation loss 0.3363668918609619
Epoch 74, Training loss 0.35541170835494995, Validation loss 0.33629536628723145
Epoch 75, Training loss 0.3554096221923828, Validation loss 0.336385041475296
Epoch 76, Training loss 0.3553798794746399, Validation loss 0.3366064131259918
Epoch 77, Training loss 0.355339914560318, Validation loss 0.33689725399017334
Epoch 78, Training loss 0.35531502962112427, Validation loss 0.3371676206588745
Epoch 79, Training loss 0.35531070828437805, Validation loss 0.3373277187347412
Epoch 80, Training loss 0.35531044006347656, Validation loss 0.33732736110687256
Epoch 81, Training loss 0.35529693961143494, Validation loss 0.33717843890190125
Epoch 82, Training loss 0.3552703559398651, Validation loss 0.33694517612457275
Epoch 83, Training loss 0.35524487495422363, Validation loss 0.3367106020450592
Epoch 84, Training loss 0.3552315831184387, Validation loss 0.33654287457466125
Epoch 85, Training loss 0.35522666573524475, Validation loss 0.3364787995815277
Epoch 86, Training loss 0.3552181124687195, Validation loss 0.3365233838558197
Epoch 87, Training loss 0.35520005226135254, Validation loss 0.3366541266441345
Epoch 88, Training loss 0.35517844557762146, Validation loss 0.33682578802108765
Epoch 89, Training loss 0.3551623523235321, Validation loss 0.3369801938533783
Epoch 90, Training loss 0.35515353083610535, Validation loss 0.3370654284954071
Epoch 91, Training loss 0.3551456332206726, Validation loss 0.3370566666126251

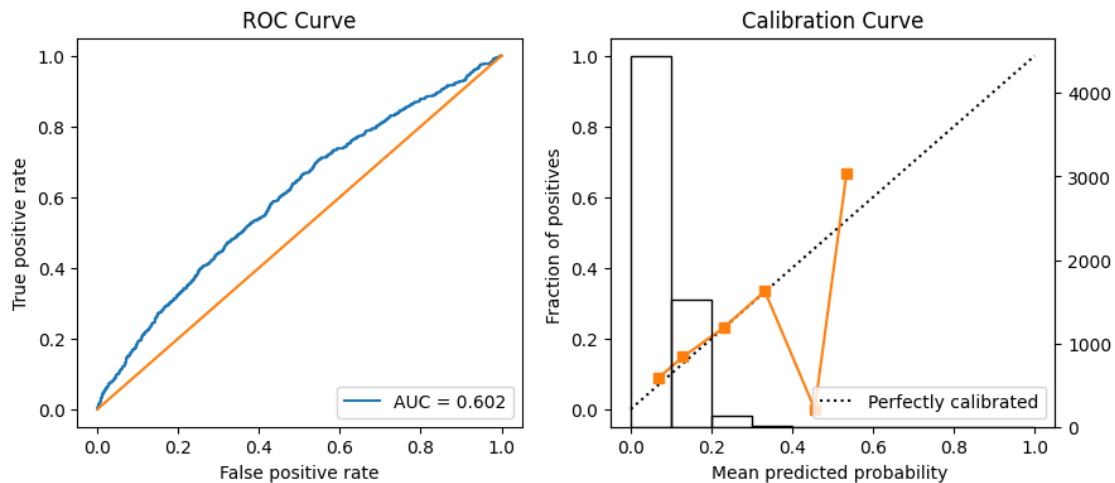
Epoch 92, Training loss 0.355133056640625, Validation loss 0.3369654417037964
 Epoch 93, Training loss 0.3551167845726013, Validation loss 0.33683180809020996
 Epoch 94, Training loss 0.3551023602485657, Validation loss 0.3367052376270294
 Epoch 95, Training loss 0.3550926744937897, Validation loss 0.3366256356239319
 Epoch 96, Training loss 0.35508519411087036, Validation loss 0.33661308884620667
 Epoch 97, Training loss 0.3550756573677063, Validation loss 0.336665540933609
 Epoch 98, Training loss 0.3550633192062378, Validation loss 0.3367617130279541
 Epoch 99, Training loss 0.35505130887031555, Validation loss 0.3368672728538513
 Epoch 100, Training loss 0.35504215955734253, Validation loss
 0.33694565296173096
 Epoch 101, Training loss 0.3550351560115814, Validation loss 0.3369717597961426
 Epoch 102, Training loss 0.3550274968147278, Validation loss 0.3369414806365967
 Epoch 103, Training loss 0.3550183176994324, Validation loss 0.33687201142311096
 Epoch 104, Training loss 0.3550090193748474, Validation loss 0.3367931842803955
 Epoch 105, Training loss 0.35500141978263855, Validation loss 0.3367346525192261
 Epoch 106, Training loss 0.3549954891204834, Validation loss 0.3367157578468323
 Epoch 107, Training loss 0.3549894392490387, Validation loss 0.3367408812046051
 Epoch 108, Training loss 0.35498255491256714, Validation loss
 0.33679965138435364
 Epoch 109, Training loss 0.35497552156448364, Validation loss 0.336870938539505
 Epoch 110, Training loss 0.35496950149536133, Validation loss 0.3369302451610565
 Epoch 111, Training loss 0.35496464371681213, Validation loss 0.3369589149951935
 Epoch 112, Training loss 0.3549599349498749, Validation loss 0.33695095777511597
 Epoch 113, Training loss 0.3549547493457794, Validation loss 0.33691462874412537
 Epoch 114, Training loss 0.3549495041370392, Validation loss 0.3368677496910095
 Epoch 115, Training loss 0.35494503378868103, Validation loss
 0.33682993054389954
 Epoch 116, Training loss 0.3549412488937378, Validation loss 0.33681520819664
 Epoch 117, Training loss 0.3549376130104065, Validation loss 0.33682781457901
 Epoch 118, Training loss 0.3549337685108185, Validation loss 0.3368615210056305
 Epoch 119, Training loss 0.35492992401123047, Validation loss 0.3369028866291046
 Epoch 120, Training loss 0.35492652654647827, Validation loss
 0.33693641424179077
 Epoch 121, Training loss 0.3549236059188843, Validation loss 0.3369506001472473
 Epoch 122, Training loss 0.35492080450057983, Validation loss 0.3369424343109131
 Epoch 123, Training loss 0.35491788387298584, Validation loss 0.3369179666042328
 Epoch 124, Training loss 0.35491499304771423, Validation loss 0.3368891179561615
 Epoch 125, Training loss 0.3549124002456665, Validation loss 0.33686843514442444
 Epoch 126, Training loss 0.35491010546684265, Validation loss
 0.33686426281929016
 Epoch 127, Training loss 0.3549078106880188, Validation loss 0.3368780016899109
 Epoch 128, Training loss 0.35490548610687256, Validation loss 0.3369044065475464
 Epoch 129, Training loss 0.3549032211303711, Validation loss 0.3369336426258087
 Epoch 130, Training loss 0.35490119457244873, Validation loss 0.3369556963443756
 Epoch 131, Training loss 0.35489925742149353, Validation loss 0.3369642198085785
 Epoch 132, Training loss 0.3548973500728607, Validation loss 0.33695870637893677
 Epoch 133, Training loss 0.35489538311958313, Validation loss 0.3369443416595459
 Epoch 134, Training loss 0.3548935055732727, Validation loss 0.3369292616844177

Epoch 135, Training loss 0.3548917770385742, Validation loss 0.33692100644111633
 Epoch 136, Training loss 0.3548901379108429, Validation loss 0.3369234800338745
 Epoch 137, Training loss 0.3548884093761444, Validation loss 0.33693575859069824
 Early stopping! Epoch 137
 Best model at epoch 37

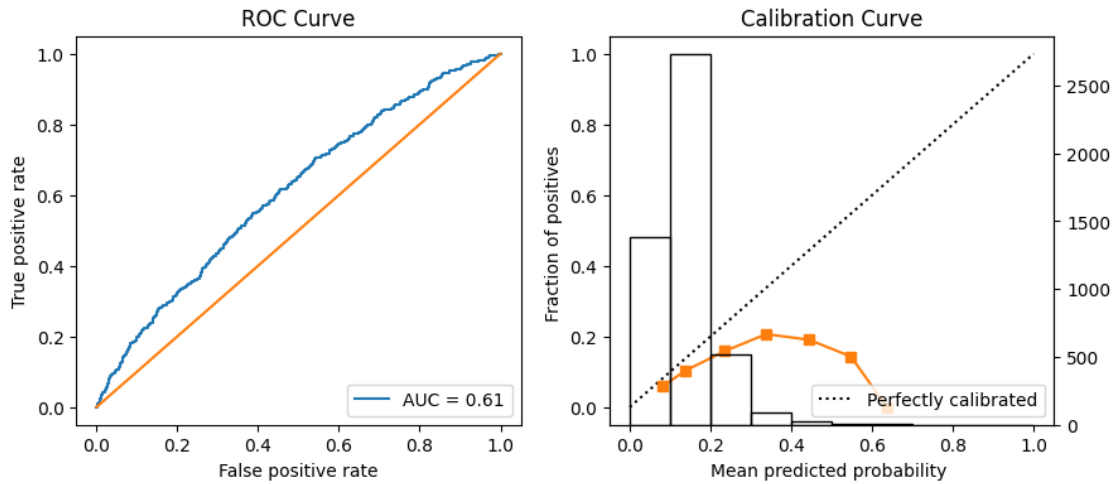
7 Evaluate the Partial Response Network

```
[104]: y_test_prn_pytorch = prn.predict(x_test_tensor, device=device).cpu().numpy()
y_val_prn_pytorch = prn.predict(x_val_tensor, device=device).cpu().numpy()

prn_metrics_test = modelStats(y_test_prn_pytorch, y_test, y_train, auc_ci=True)
modelStats(y_val_prn_pytorch, y_val, y_train, auc_ci=True)
```



```
----- Metrics -----
prevalence: 0.123
sensitivity: 0.235
specificity: 0.867
accuracy: 0.799
ppv: 0.174
auc score: 0.602
auc lower ci: 0.578
auc upper ci: 0.622
-----
```



```

----- Metrics -----
prevalence: 0.123
sensitivity: 0.639
specificity: 0.512
accuracy: 0.524
ppv: 0.126
auc score: 0.61
auc lower ci: 0.583
auc upper ci: 0.632
-----

```

```

[104]: {'prevalence': 0.123,
        'sensitivity': 0.639,
        'specificity': 0.512,
        'accuracy': 0.524,
        'ppv': 0.126,
        'auc score': 0.61,
        'auc lower ci': '0.583',
        'auc upper ci': '0.632'}

```

8 LASSO on the Partial Response Network

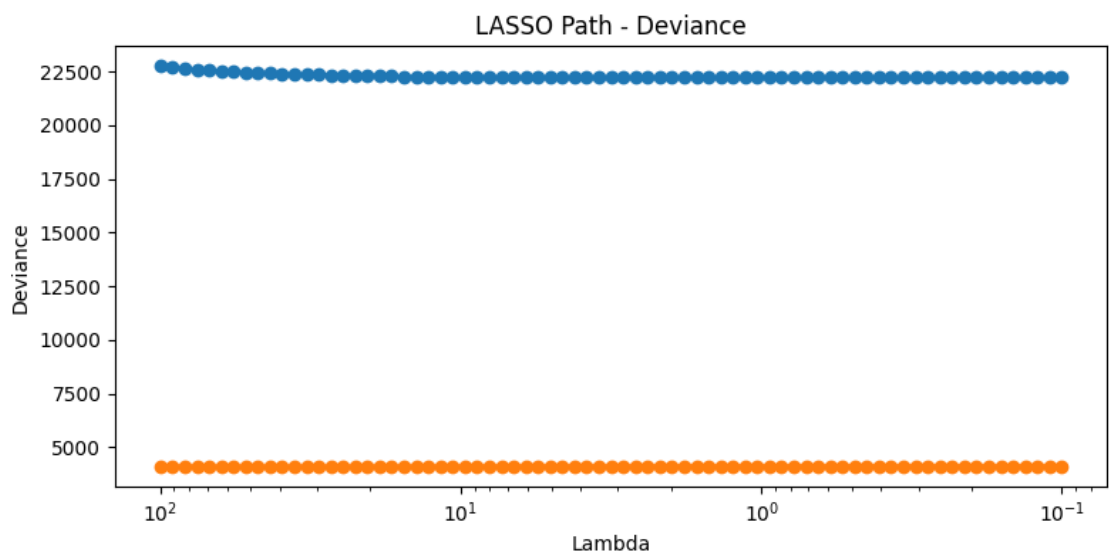
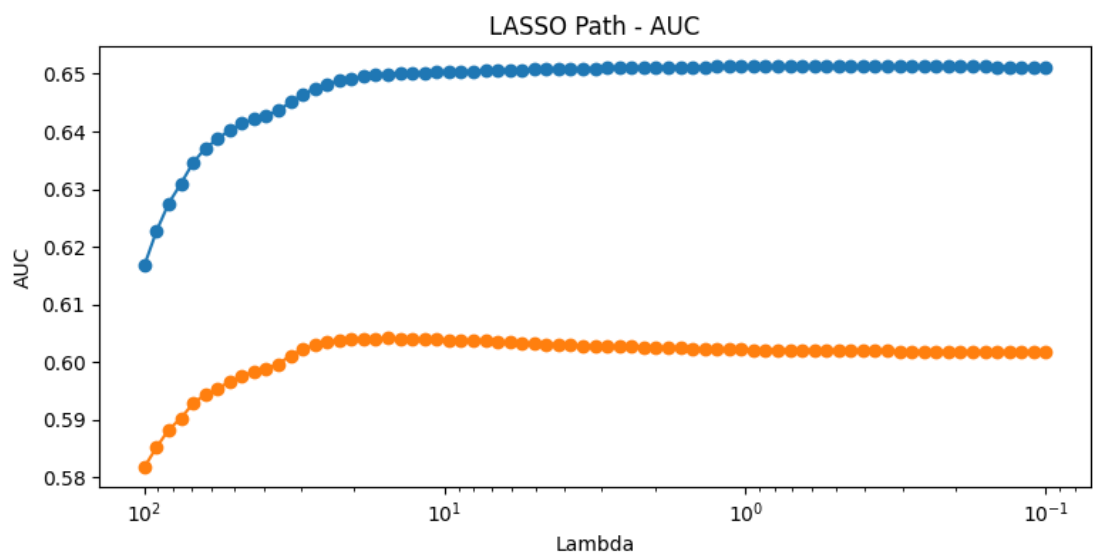
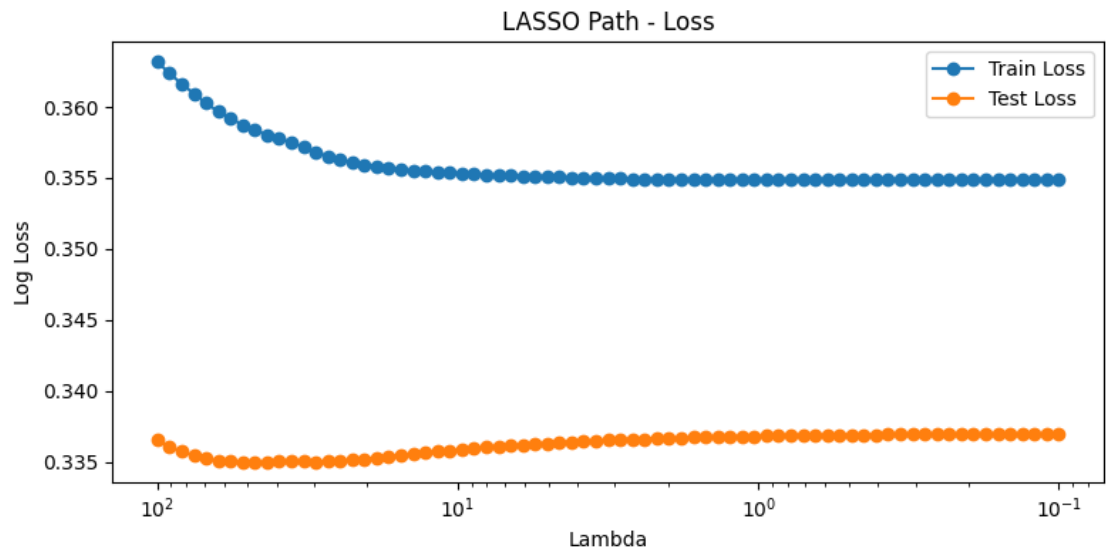
```

[105]: # Use the new partial_responses function for PRN
partial_responses_train_prn, partial_responses_test_prn, bivariate_inputs_prn =
↳ partial_responses(
    x_train_tensor,
    x_test_tensor,
    prn,
    method=method,

```

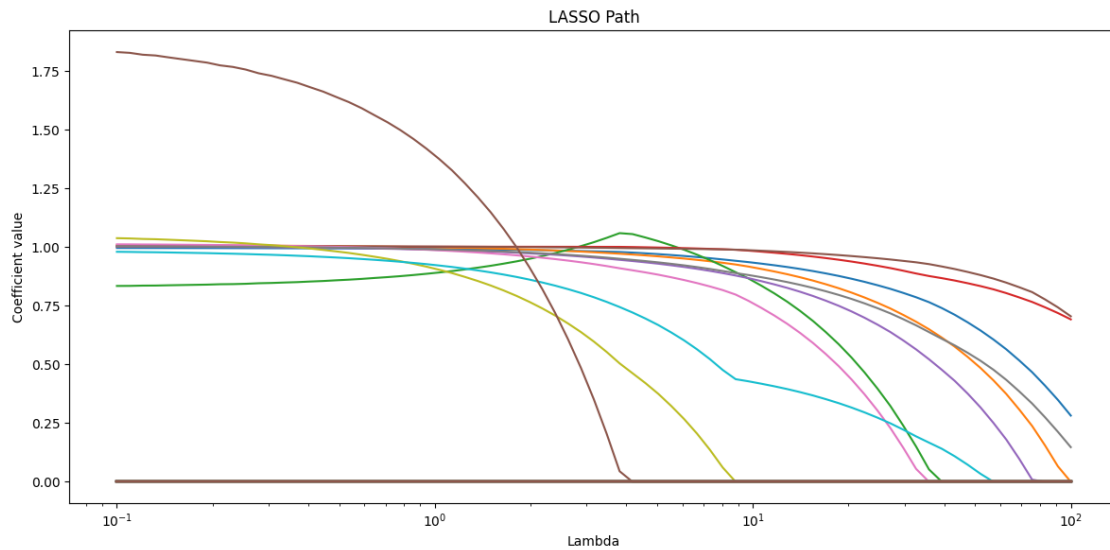
```
    device=device  
)
```

```
[106]: %reload_ext autoreload  
lasso_results_prn = lasso(  
    partial_responses_train_prn,  
    partial_responses_test_prn,  
    y_train,  
    y_test,  
    bivariate_inputs_prn,  
    feature_names=feature_names,  
    nlambdas=75,  
    min_lambda=0.1,  
    max_lambda=100,  
    tol=1e-4,  
    batch_size=2  
)
```

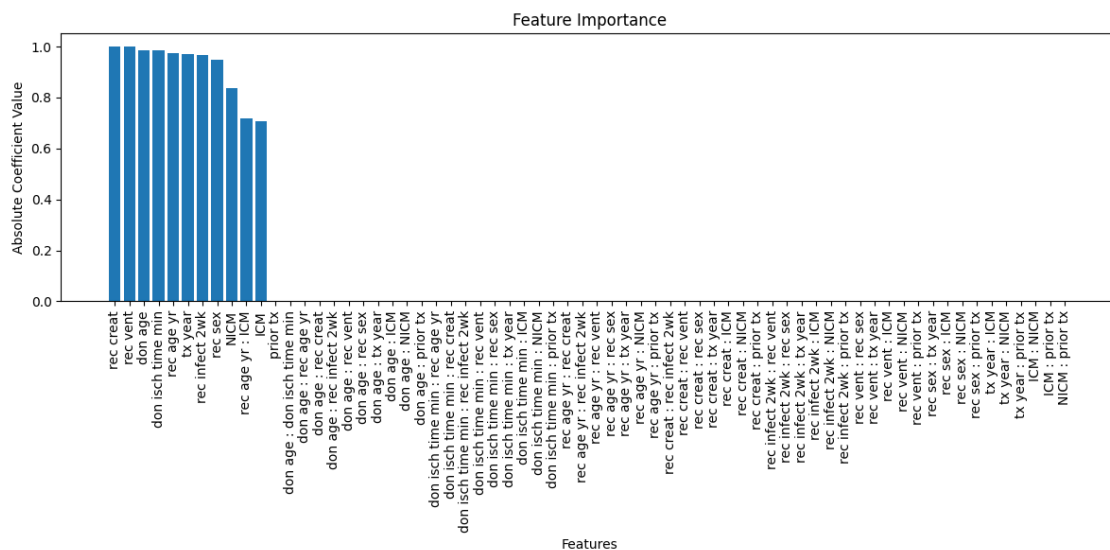


<IPython.core.display.HTML object>

```
[107]: lasso_results_prn.plot_lambda_path()
```



```
[108]: lasso_results_prn.select_lambda(40)
lasso_results_prn.plot_feature_importance()
```

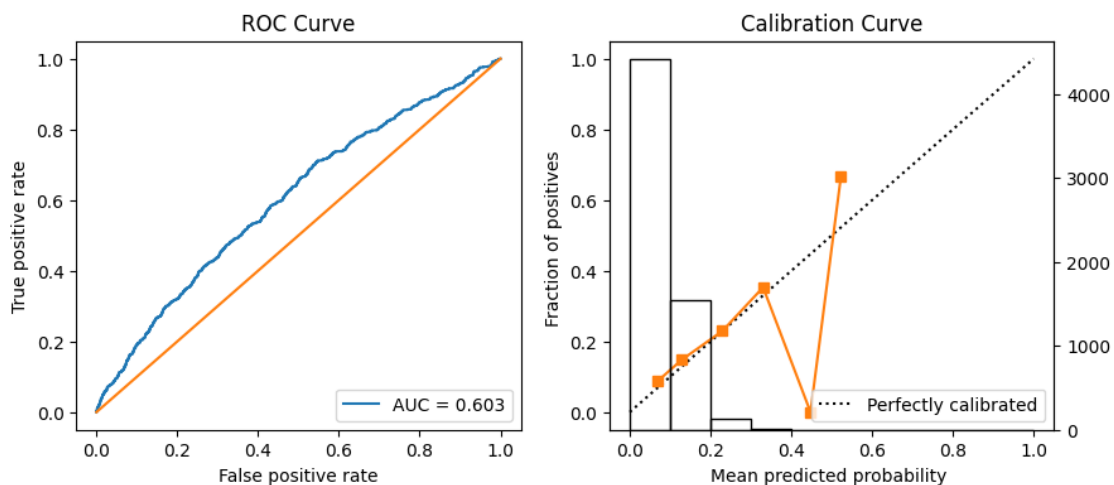


9 Validation inference

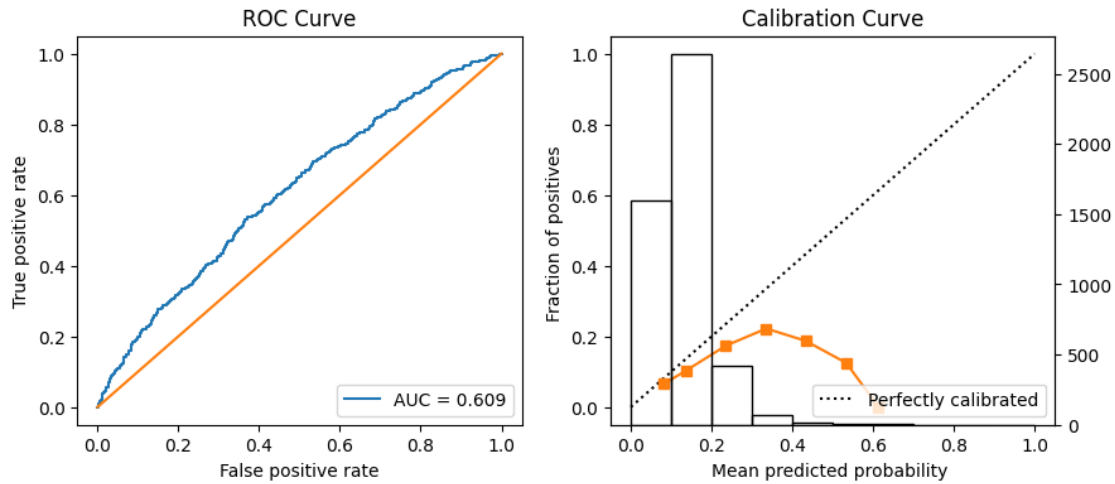
```
[109]: prn_lasso = lasso_results_prn.get_selected_model()

[110]: _, partial_responses_val_prn, bivariate_inputs_prn = partial_responses(
    x_train_tensor,
    x_val_tensor,
    prn,
    method=method,
    device=device
)

[111]: y_pred_test_prn_lasso = prn_lasso.predict_proba(partial_responses_test_prn)[: , 1]
      y_pred_val_prn_lasso = prn_lasso.predict_proba(partial_responses_val_prn)[: , 1]
      lasso_metrics_test = modelStats(y_pred_test_prn_lasso, y_test, y_train,
      auc_ci=True)
      modelStats(y_pred_val_prn_lasso, y_val, y_train, auc_ci=True)
```



```
----- Metrics -----
prevalence: 0.123
sensitivity: 0.238
specificity: 0.867
accuracy: 0.8
ppv: 0.177
auc score: 0.603
auc lower ci: 0.578
auc upper ci: 0.626
-----
```



```

----- Metrics -----
prevalence: 0.123
sensitivity: 0.594
specificity: 0.558
accuracy: 0.562
ppv: 0.129
auc score: 0.609
auc lower ci: 0.586
auc upper ci: 0.631
-----

```

```

[111]: {'prevalence': 0.123,
        'sensitivity': 0.594,
        'specificity': 0.558,
        'accuracy': 0.562,
        'ppv': 0.129,
        'auc score': 0.609,
        'auc lower ci': '0.586',
        'auc upper ci': '0.631'}

```

10 Partial Response Network Nomogram

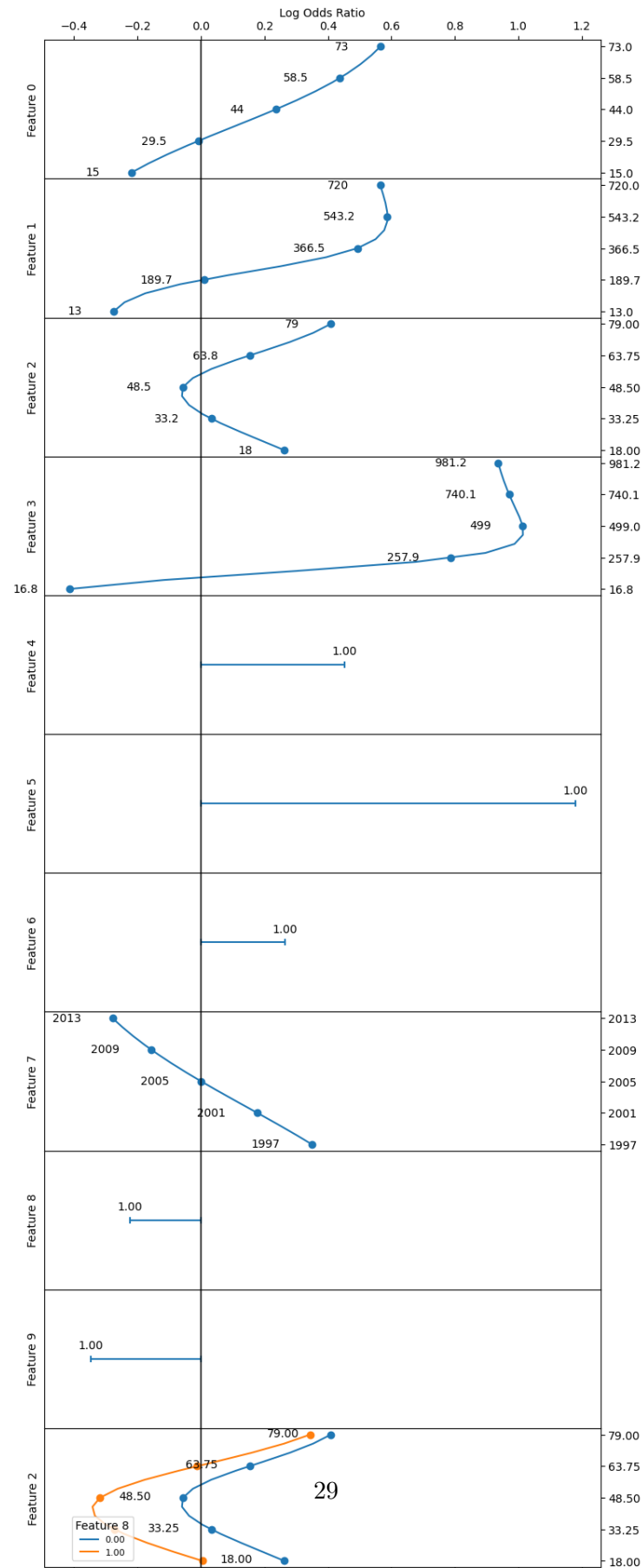
```

[112]: nomogram(
        lasso_results_prn,
        x_train_tensor,
        x_train0.median().values,
        x_train0.std().values,
        prn,
        n_steps=15,

```

```
sd_scale=2,  
method=method,  
device=device,  
categorical_threshold=15  
);
```

Nomogram of univariate and mixed bivariate partial responses

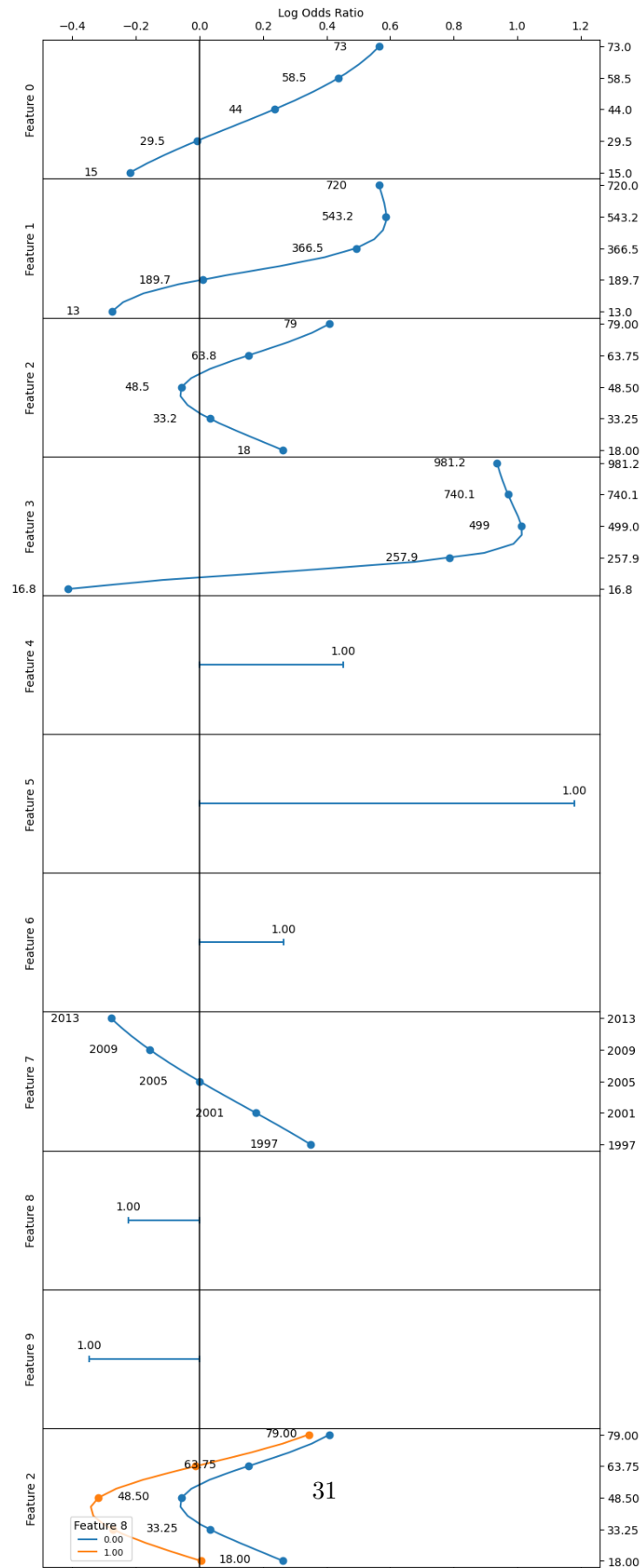


10.1 Nomogram with more

TODO: modify `lasso_results` to include possibility to select all features, all univariate, all bivariate (e.g. for plotting).

```
[113]: # test with more features  
# todo add all features method to lasso_results object.  
lasso_results_prn.select_lambda(74)  
nomogram(  
    lasso_results_prn,  
    x_train_tensor,  
    x_train0.median().values,  
    x_train0.std().values,  
    prn,  
    n_steps=15,  
    sd_scale=2,  
    method=method,  
    device=device,  
    categorical_threshold=15  
);
```

Nomogram of univariate and mixed bivariate partial responses



[]: